

International Institute of Information Technology
Bangalore

Software Production Engineering
CS 816

Mini Project Report

Scientific Calculator with DevOps

Shikhar Bhadreshkumar Mutta (MT2025114)

Github Link
Docker hub Link

March 6, 2026



Contents

1	Introduction: What and Why of DevOps?	2
2	Tools Used	2
3	Jenkinsfile — Stage-by-Stage Explanation	3
3.1	Global Configuration	3
3.2	Stage 1: Checkout	3
3.3	Stage 2: Build	4
3.4	Stage 3: Test	5
3.5	Stage 4: Package	6
3.6	Stage 5: Docker Build	6
3.7	Stage 6: Docker Push	7
3.8	Stage 7: Deploy with Ansible	8
3.9	Post — Notifications & Cleanup	9
4	Test Results & Observations	10
5	System Architecture & Resource Pruning	11
5.1	Multi-Stage Dockerfile with jlink	11
5.2	Post-Build Disk Cleanup	12
6	Application Output	13

1 Introduction: What and Why of DevOps?

What is DevOps?

DevOps is a set of cultural philosophies, practices, and tools that combines software development (Dev) and IT operations (Ops). It aims to shorten the systems development life cycle while delivering features, fixes, and updates frequently in close alignment with business objectives.

Why DevOps?

In the context of this scientific calculator project, manual compilation, testing, and deployment of the Java application are prone to human error and are highly time-consuming. Adopting DevOps methodologies automates these processes. A Continuous Integration and Continuous Deployment (CI/CD) pipeline ensures that every time a developer commits code to the repository, it is instantly built, rigorously tested against edge cases, smoothly containerized, and deployed to production servers without manual downtime.

2 Tools Used

The complete end-to-end automated pipeline relies on the following DevOps toolchain:

- **Source Control Management:** GitHub
- **Build Tool:** Maven
- **Testing Framework:** JUnit
- **Continuous Integration:** Jenkins
- **Containerization:** Docker
- **Configuration Management:** Ansible

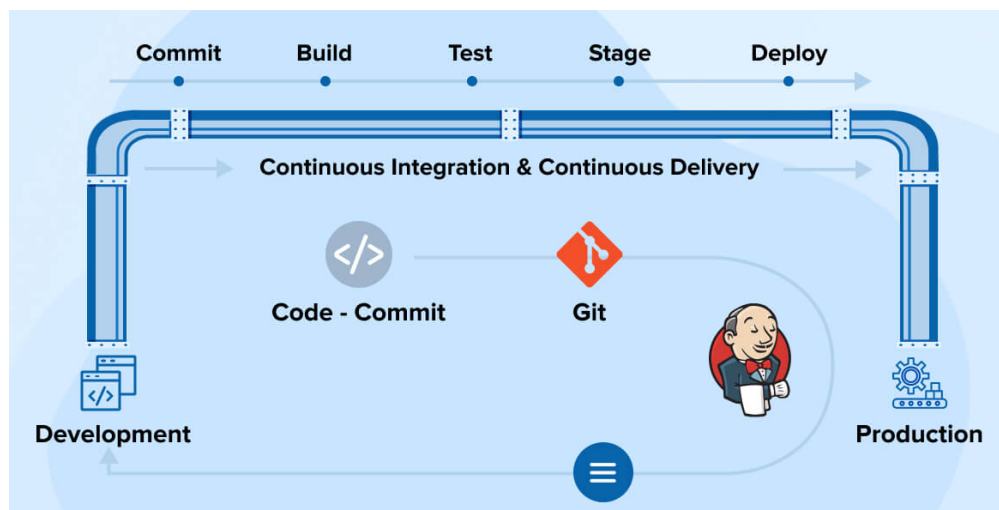


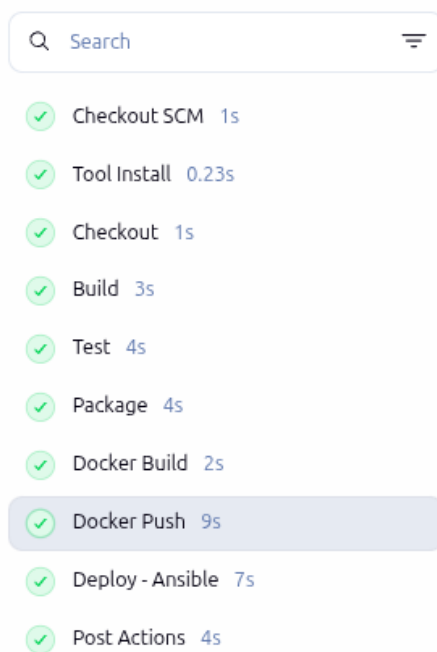
Figure 1: High-level architectural diagram of the CI/CD pipeline

3 Jenkinsfile — Stage-by-Stage Explanation



Figure 2: Jenkins Pipeline Stages

3.1 Global Configuration



`agent any` allows the pipeline to run on any Jenkins executor. Environment variables centralise the Docker image name. `DOCKER_BUILDKIT=0` forces the legacy Docker builder to avoid known socket-hang issues with BuildKit inside Jenkins.

Figure 3: Time taken by stages

3.2 Stage 1: Checkout

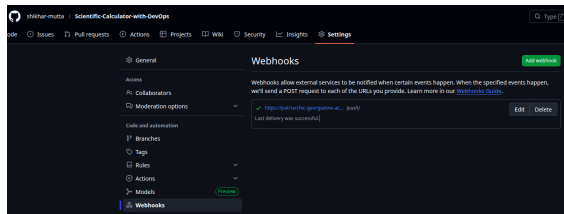
```
stage('Checkout') {
  steps {
    script {
      try {
        git branch: 'main',
            url: 'https://github.com/shikhar-mutta/' +
                'Scientific-Calculator-with-DevOps.git'
      } catch (Exception e) {
        currentBuild.description = 'Failed Stage: Checkout\n' +
            'Reason: ' + e.getMessage()
      }
    }
  }
}
```

```

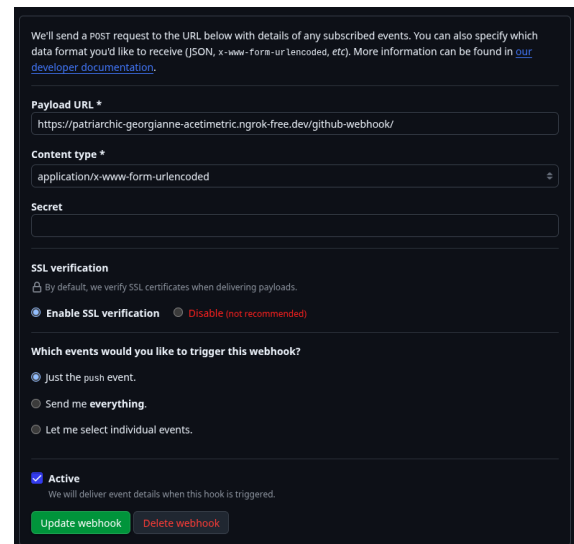
    }
    throw e
  }
}
}
}

```

Clones the main branch from GitHub into the Jenkins workspace. Every stage wraps its logic in try-catch — on failure, a descriptive message is stored in `currentBuild.description` which appears in the failure email, allowing the developer to immediately know which stage failed and why.



(a) Webhook at git project setting



(b) Webhook Configuration

Figure 4: GitHub Push Webhook

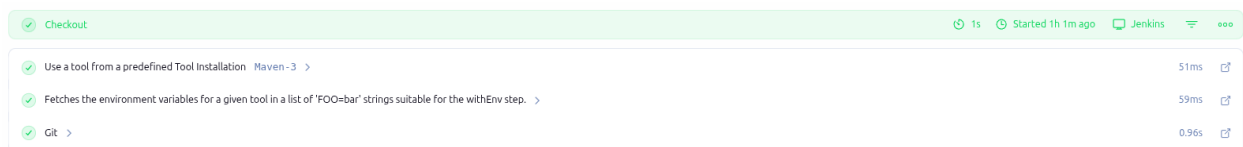


Figure 5: Jenkins Checkout Stage

3.3 Stage 2: Build

```

stage('Build') {
    steps {
        script {
            try {
                sh 'mvn clean compile -q'
            } catch (Exception e) {

```

```

        currentBuild.description =
            'Failed Stage: Build\nReason: Maven compilation failed.'
        throw e
    }
}
}
}
}

```

`mvn clean compile -q` wipes previous build output and compiles all Java source files. The `-q` (quiet) flag suppresses verbose Maven output. Fails fast on syntax errors, preventing wasteful downstream steps from executing.

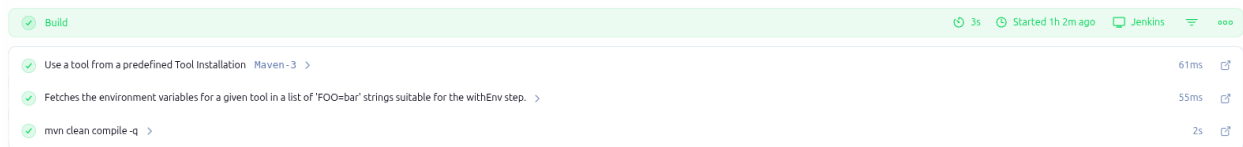


Figure 6: Jenkins Console Output showing successful Maven Compilation

3.4 Stage 3: Test

```

stage('Test') {
    steps {
        script {
            try {
                sh 'mvn test'
            } catch (Exception e) {
                currentBuild.description =
                    'Failed Stage: Test\n' +
                    'Reason: One or more JUnit tests failed.'
                throw e
            }
        }
    }
}

```

Runs the full JUnit 5 suite (37 tests) via Maven Surefire plugin. Tests cover all 8 operations including boundary cases and exception scenarios. Any failure marks the build as FAILURE and prevents broken code from being containerised and deployed.

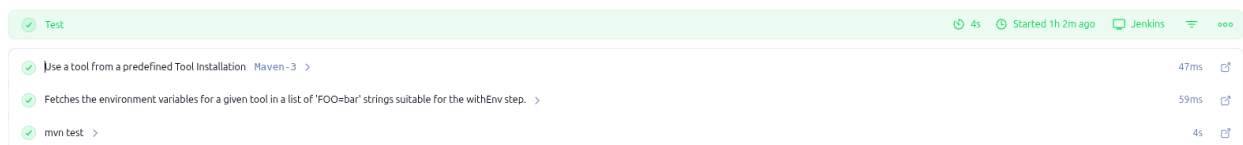


Figure 7: Jenkins Console Output displaying JUnit execution

3.5 Stage 4: Package

```
stage('Package') {
    steps {
        script {
            try {
                sh 'mvn package -DskipTests -q'
            } catch (Exception e) {
                currentBuild.description =
                    'Failed Stage: Package\n' +
                    'Reason: Failed to package the JAR file.'
                throw e
            }
        }
    }
}
```

The compiled classes are bundled into a runnable JAR (`scientific-calculator-1.0-SNAPSHOT.jar`) within the `target/` directory. Tests are skipped (`-DskipTests`) since they have already passed in Stage 3. This JAR is then copied into the Docker image during the next stage.

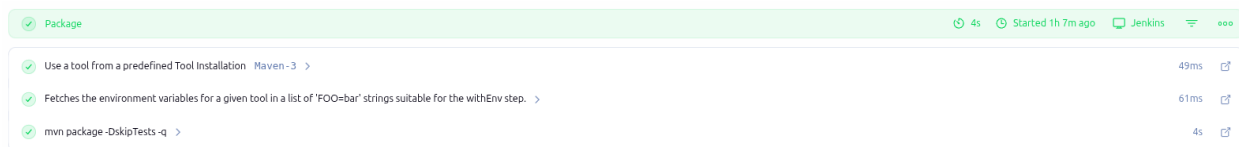


Figure 8: Jenkins Console Output showing Maven JAR packaging

3.6 Stage 5: Docker Build

```
stage('Docker Build') {
    options {
        timeout(time: 15, unit: 'MINUTES') // Kill stage if Docker hangs
    }
    steps {
        script {
            try {
                // Verify daemon reachable
                sh 'docker info > /dev/null 2>&1'
                sh "docker build -t ${DOCKER_IMAGE}:${DOCKER_TAG} ."
            } catch (Exception e) {
                currentBuild.description =
                    'Failed Stage: Docker Build\n' +
                    'Reason: Docker image build failed.'
                throw e
            }
        }
    }
}
```

```

    }
  }
}

```

A 15-minute timeout guards against Docker daemon hangs. The pre-check `docker info` verifies the daemon is responsive. A 3-stage Dockerfile is used:

- **Stage 1:** Maven builder — downloads dependencies, compiles and packages the JAR
- **Stage 2:** `jlink` — creates a custom minimal JRE including only the `java.base`, `java.logging`, and `java.desktop` modules (17.9 MB layer)
- **Stage 3:** Alpine 3.19.9 (3.26 MB base) + custom JRE + JAR (4.18 KB) — final compressed image **21.17 MB**

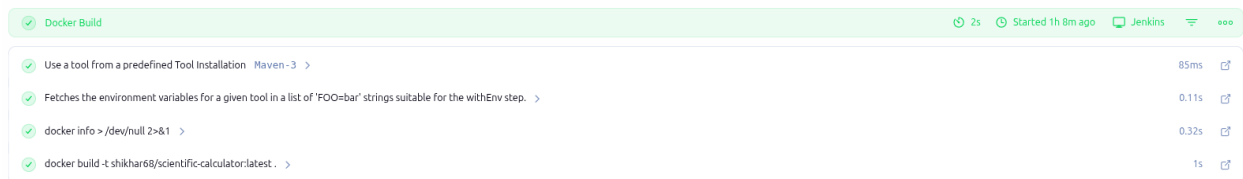


Figure 9: Jenkins Console Output showing Docker image build and layer caching

3.7 Stage 6: Docker Push

```

stage('Docker Push') {
    steps {
        script {
            try {
                withCredentials([usernamePassword(
                    credentialsId: 'docker-hub-credentials',
                    usernameVariable: 'DOCKER_USER',
                    passwordVariable: 'DOCKER_PASS'
                )]) {
                    sh "echo $DOCKER_PASS | docker login " +
                        "-u $DOCKER_USER --password-stdin"
                    sh "docker push ${DOCKER_IMAGE}:${DOCKER_TAG}"
                    sh 'docker logout'
                }
            } catch (Exception e) { ... }
        }
    }
}

```


Docker Hub credentials are stored in Jenkins Credential Manager (ID: `docker-hub-credentials`) and never appear in plain text. The password is passed via `-password-stdin` to avoid shell history exposure. `docker logout` clears credentials from the Jenkins agent after the push.

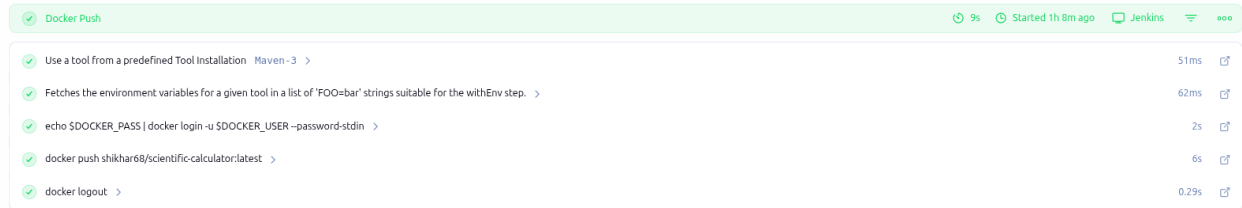


Figure 10: Jenkins Console Output showing Docker image push to DockerHub

3.8 Stage 7: Deploy with Ansible

```
stage('Deploy - Ansible') {
    steps {
        script {
            try {
                sh 'ansible-playbook -i inventory.ini deploy.yml'
            } catch (Exception e) {
                currentBuild.description =
                    'Failed Stage: Deploy\nReason: Ansible deployment failed.'
                throw e
            }
        }
    }
}
```

The Ansible playbook (`deploy.yml`) executes 6 tasks on localhost via `ansible_connection=local`:

- [1] Log the old container status (ID, Image, uptime) before update
- [2] Pull latest image from DockerHub (`docker pull`)
- [3] Force-remove the existing container (`docker rm -f`)
- [4] Start the new container: `docker run -d -i -restart unless-stopped (-i keeps stdin open since the app is CLI-driven)`
- [5] Verify the new container is running (`docker ps`)
- [6] Print new container short ID for confirmation

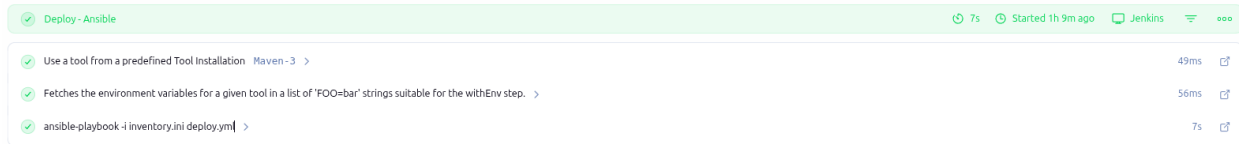


Figure 11: Jenkins Console Output showing Ansible playbook deployment execution

3.9 Post — Notifications & Cleanup

```
post {
    success {
        mail to: 'shikharmutta67@gmail.com',
              subject: "SUCCESS: Scientific Calculator Pipeline " +
                      "- Build #${env.BUILD_NUMBER}",
              body: "Pipeline completed successfully..."
    }
    failure {
        script {
            def info = currentBuild.description ?:
                'Failed Stage: SCM Checkout...'
            mail to: 'shikharmutta67@gmail.com',
                  subject: "FAILURE: Pipeline - Build " +
                          "#${env.BUILD_NUMBER}",
                  body: "Pipeline failed.\n\n${info}\n\n" +
                       "Console: ${env.BUILD_URL}console"
        }
    }
    always {
        sh 'docker image prune -f || true'    // Clean dangling images
    }
}
```

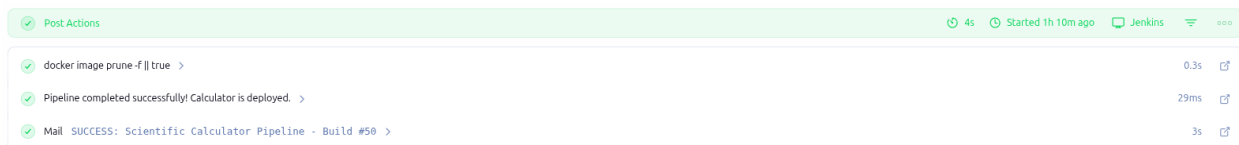


Figure 12: Jenkins Console Output showing post-build actions and email notifications

Success and failure emails are sent via Jenkins Mailer plugin. Failure emails include the exact stage name and reason captured by each catch block. Dangling Docker images (intermediate build layers) are always pruned to conserve disk space.

4 Test Results & Observations

All 37 JUnit 5 unit tests pass consistently across every pipeline run. Tests cover both happy-path and error/exception scenarios for all 8 operations. Maven Surefire plugin reports: Tests run: 37, Failures: 0, Errors: 0, Skipped: 0

Test Group	Tests	What is Tested	Result
Addition	4	Positive, negative, zero, decimal inputs	✓ PASS
Subtraction	4	Positive result, negative result, zero, same numbers	✓ PASS
Multiplication	4	Positive, by zero, two negatives, mixed signs	✓ PASS
Division	4	Positive, negative, zero numerator, divide-by-zero throws	✓ PASS
Power	5	Pos/neg/zero/fractional exponent, base zero	✓ PASS
Square Root	4	Positive, zero, decimal ($\sqrt{2}$), negative input throws	✓ PASS
Natural Log	5	$\ln(1) = 0$, $\ln(e) = 1$, $\ln(10)$, $\ln(0)$ throws, $\ln(-x)$ throws	✓ PASS
Factorial	7	$0!$, $1!$, $5!$, $10!$, $20!$, negative throws, $n > 20$ throws	✓ PASS
TOTAL	37	—	✓ ALL PASS

Table 1: Detailed Breakdown of JUnit Test Execution

Key Observations:

- **Exception coverage:** We utilize `assertThrows` (e.g., `IllegalArgumentException.class`) to verify correct error handling for invalid inputs (such as divide-by-zero, square root of negative, logarithm of non-positive, and factorial overflow).
- **Floating-point precision:** All double comparisons use a delta of $1e-9$ to handle IEEE 754 floating-point rounding correctly.
- **Stage separation:** Tests are a separate pipeline stage from Build — a test failure gives clear feedback without rerequiring recompilation.
- **Performance:** The test stage completes in approximately 2–4 seconds due to pure unit tests with no I/O or network calls.

Maven Surefire Report – ScientificCalculatorTest						✓ BUILD SUCCESS
Tests: 37	Failures: 0	Errors: 0	Skipped: 0	Time: 0.114s	JUnit: 5.9.3	Java: 21.0.10

Figure 13: Detailed Output of the Test Cases or Surefire Reports

5 System Architecture & Resource Pruning

Keeping the Docker image small and the build environment clean are critical for a sustainable CI/CD pipeline. This project employs two complementary strategies: a multi-stage Dockerfile with `jlink` to minimise the production image, and automated post-build cleanup to reclaim disk space on the Jenkins agent.

5.1 Multi-Stage Dockerfile with `jlink`

The Dockerfile uses three distinct stages to produce a minimal, production-ready container image:

1. **Stage 1 — Maven Builder** (`maven:3.9-eclipse-temurin-17`): Dependencies are downloaded and cached via `mvn dependency:go-offline`. The application is then compiled and packaged into a runnable JAR (`scientificcalculator1.0-SNAPSHOT.jar`) using `mvn clean package -DskipTests -o -q`. Separating the dependency download from the build step enables Docker layer caching — subsequent builds skip the download if `pom.xml` has not changed.
2. **Stage 2 — Custom JRE via `jlink`** (`eclipse-temurin:17-jdk-alpine`): The Alpine JDK variant is used so that the output is `musl`-compatible. `jlink` creates a stripped-down JRE containing only the modules the application requires:

```
jlink --add-modules java.base,java.logging \  
      --strip-debug --no-man-pages \  
      --no-header-files --compress=2 \  
      --output /custom-jre
```

Flags such as `-strip-debug`, `-no-man-pages`, and `-compress=2` further reduce the JRE size from ~ 300 MB (full JDK) down to ~ 18 MB.

3. **Stage 3 — Alpine Runtime** (`alpine:3.19`): The final image copies only the custom JRE and the application JAR onto a bare Alpine 3.19 base (3.26 MB). A non-root user (`appuser`) is created for security, and the entrypoint is set to `java -jar app.jar`.

The resulting production image is approximately **21 MB** — a $\sim 96\%$ reduction compared to a naive single-stage build using the full JDK image (~ 500 MB).

Layer	Size	Purpose
Alpine 3.19 base	3.26 MB	Minimal OS (musl libc, BusyBox)
Custom JRE (jlink)	~ 17.9 MB	java.base + java.logging only
Application JAR	4.18 KB	Compiled calculator classes
Total	~ 21 MB	Final compressed image

Table 2: Docker image layer breakdown

5.2 Post-Build Disk Cleanup

Every pipeline run — whether it succeeds or fails — triggers a post-build action in the `always` block of the Jenkinsfile:

```
post {  
    always {  
        sh 'docker image prune -f || true'  
    }  
}
```

`docker image prune -f` removes all *dangling* images (untagged intermediate layers left behind by multi-stage builds). The `|| true` suffix ensures the pipeline does not fail if there are no images to prune. Over multiple builds, this prevents unbounded disk consumption on the Jenkins agent.

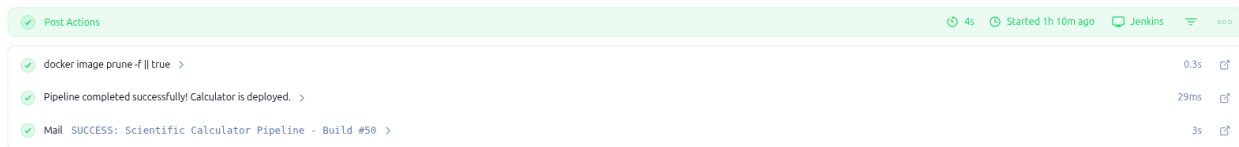


Figure 14: Jenkins post-build actions: Docker image pruning and email notifications

6 Application Output

Because this is a command-line interface application executed securely inside a microservice container, it is provisioned completely detached from the local machine's host OS vulnerabilities. End users access it via standard interactive TTY hooks.

```
shikhar@shikhar-pc:~/Sem 2/SPE/Mini Project/Scientific-Calculator-with-DevOps$ docker attach scientific-calculator

=====
      SCIENTIFIC CALCULATOR
      SPE Mini Project
=====

--- Menu ---
1. Add
2. Subtract
3. Multiply
4. Divide
5. Power
6. Square Root
7. Logarithm
8. Factorial
9. Exit
Choose an option (1-9):
```

Figure 15: Execution of `docker attach scientific-calculator` showing the calculator UI

```
shikhar@shikhar-pc:~/Sem 2/SPE/Mini Project/Scientific-Calculator-with-DevOps$ docker run -it shikhar68/scientific-calculator:latest

=====
      SCIENTIFIC CALCULATOR
      SPE Mini Project
=====

--- Menu ---
1. Add
2. Subtract
3. Multiply
4. Divide
5. Power
6. Square Root
7. Logarithm
8. Factorial
9. Exit
```

Figure 16: Execution of `docker run -it shikhar68/scientific-calculator:latest` showing the calculator UI

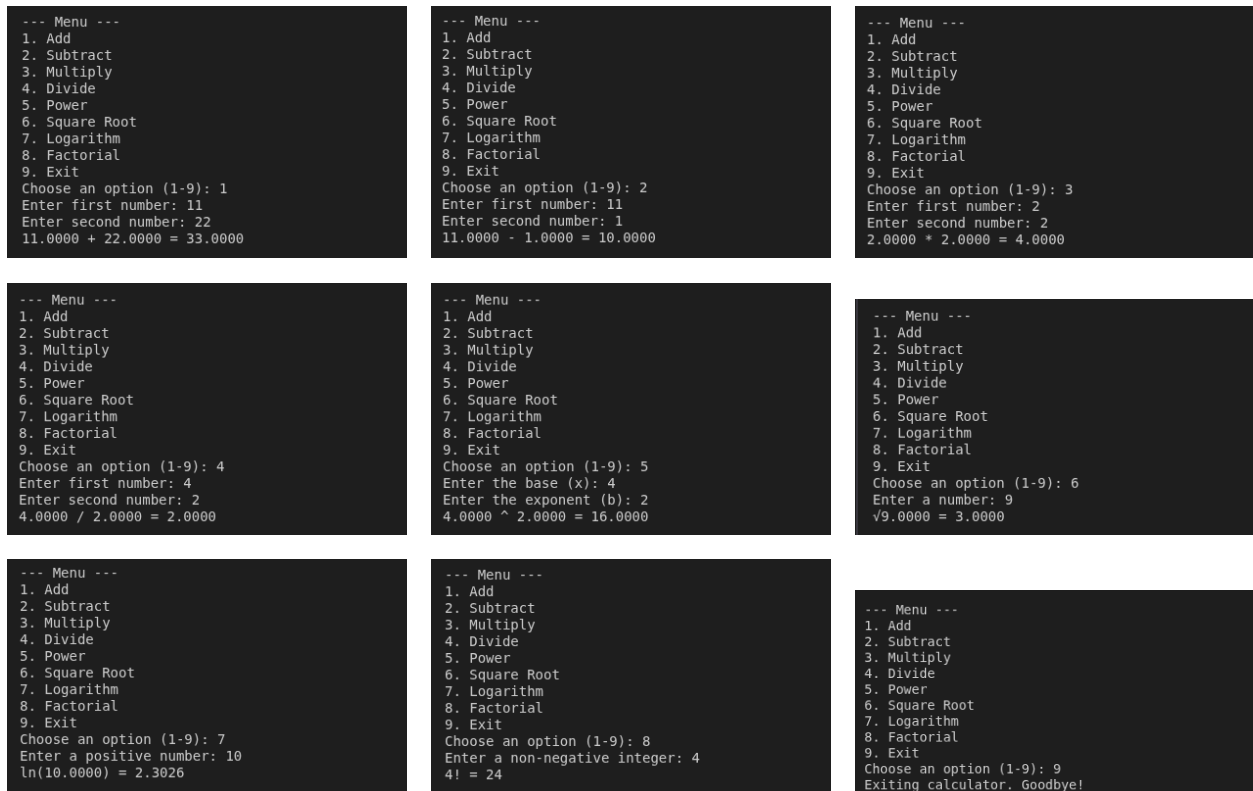


Figure 17: Mathematical operations returning accurate answers inside the terminal